

Ducks Unlimited University Chapters ETL Pipeline

AIM: The aim of this project is to build an end-to-end automated ETL (Extract, Transform, Load) pipeline that extracts university chapter data for Ducks Unlimited from a public ArcGIS API endpoint, validates and transforms the records into a clean schema, and securely loads them downstream into Google BigQuery.

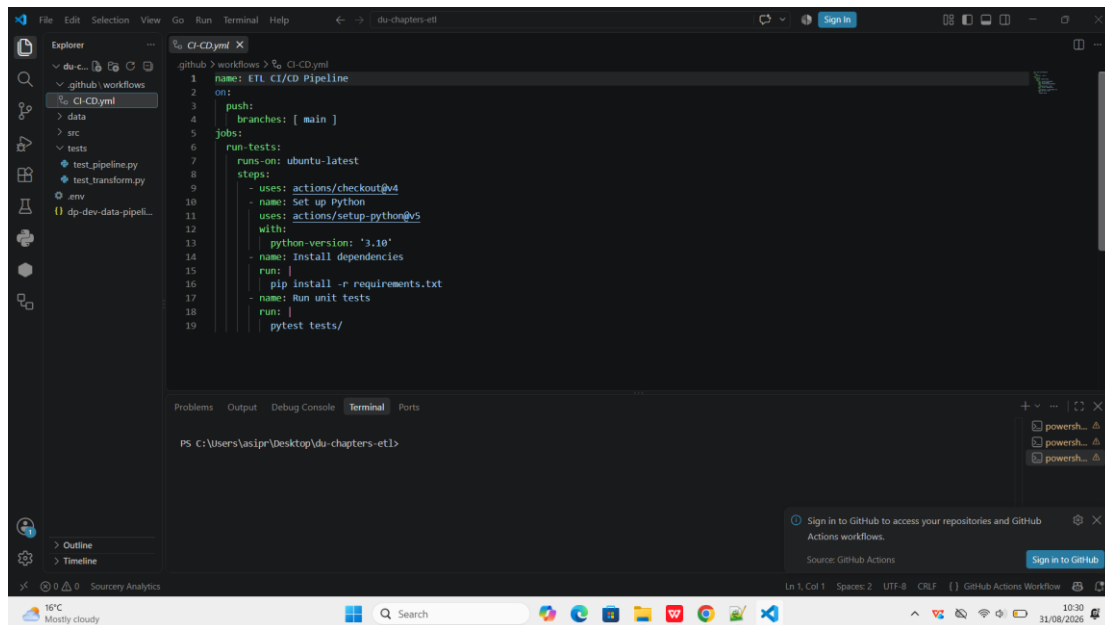
Key objectives include:

- 1.) Automated Data Integration: Programmatically fetching university chapter records using Python instead of manual data collection.
- 2.) Data Quality & Transformation: Parsing nested JSON structures and spatial attributes to ensure uniform data formats, accurate coordinates, and explicit schema constraints.
- 3.) Cloud Storage: Persisting structured data reliably inside Google BigQuery for reporting and analysis.
- 4.) Software Engineering Best Practices: Incorporating modular script design, automated unit testing with `pytest`, and continuous integration/continuous deployment (CI/CD) workflows via GitHub Actions.

Below are the source file names and target table which I used to migrate the data:

Component Type	Name / Identifier
Source File	src/src_extract.py
Source File	src/src_transform.py
Source File	src/src_bigquery_load.py
Source File	src/src_pipeline.py
Testing File	tests/test_pipeline.py
Testing File	tests/test_transform.py
CI/CD Config	.github/workflows/CI-CD.yml
Cloud Target	Google BigQuery Project (dp-dev-data-pipeline)
Cloud Target	BigQuery Dataset (etl_pipeline)
Cloud Target	BigQuery Table (university_chapters)

Step 1:- This YAML file configures a GitHub Actions workflow named "ETL CI/CD Pipeline" that automates testing for myrepository.



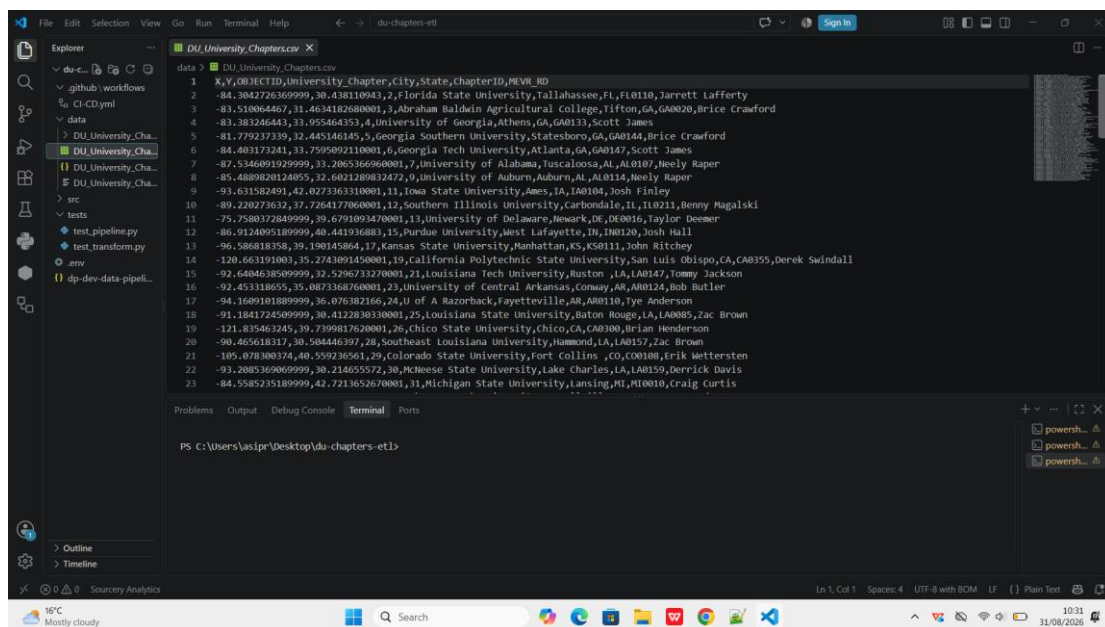
The screenshot shows a Visual Studio Code editor with a file explorer on the left and a code editor in the center. The file explorer shows a project structure with folders like 'data', 'src', 'tests', and files like 'test_pipeline.py', 'test_transform.py', and 'env'. The code editor displays a GitHub Actions workflow file named 'CI-CD.yml' with the following content:

```
1 name: ETL CI/CD Pipeline
2 on:
3   push:
4     branches: [ main ]
5 jobs:
6   run-tests:
7     runs-on: ubuntu-latest
8     steps:
9       - uses: actions/checkout@v4
10      - name: Set up Python
11        uses: actions/setup-python@v5
12        with:
13          python-version: '3.10'
14      - name: Install dependencies
15        run: |
16          pip install -r requirements.txt
17      - name: Run unit tests
18        run: |
19          pytest tests/
```

The terminal at the bottom shows the command 'PS C:\Users\asip\Desktop\du-chapters-eti>'.

Step 2:- Inside the data folders I have real time data in CSV and other formats

CSV:-

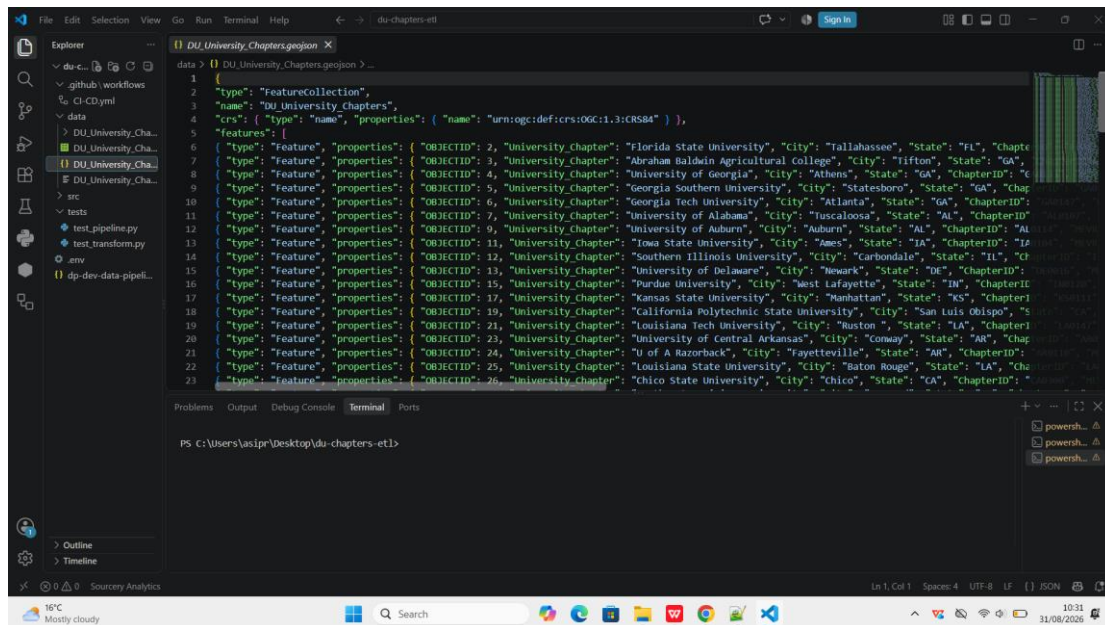


The screenshot shows a Visual Studio Code editor with a file explorer on the left and a code editor in the center. The file explorer shows a project structure with folders like 'data', 'src', 'tests', and files like 'test_pipeline.py', 'test_transform.py', and 'env'. The code editor displays a CSV file named 'DU_University_Chapters.csv' with the following content:

```
1 X,Y,OBJECTID,University_chapter,City,State,chapterID,NAME_RD
2 -84.3842726369999,30.438110943,2,Florida State University,Tallahassee,FL,FL0110,Jarrett Lafferty
3 -83.510064467,31.4634182680001,3,Abraham Baldwin Agricultural College,Tifton,GA,GA0020,Brice Crawford
4 -83.382306441,33.058464353,4,University of Georgia,Athens,GA,GA0133,Scott James
5 -81.779237339,32.442146145,5,Georgia Southern University,Statesboro,GA,GA0144,Brice Crawford
6 -84.403173241,33.7595992110001,6,Georgia Tech University,Atlanta,GA,GA0147,Scott James
7 -87.5346091929999,33.2065366960001,7,University of Alabama,Tuscaloosa,AL,AL0107,Neely Raper
8 -85.4889820124055,32.6021289832472,9,University of Auburn,Auburn,AL,AL0114,Neely Raper
9 -93.631582491,42.0273363310001,11,Iowa State University,Ames,IA,IA0104,Josh Finley
10 -89.220273632,37.7264177060001,12,Southern Illinois University,Carbondale,IL,IL0211,Benny Magalski
11 -75.7580172849999,39.6791093470001,13,University of Delaware,Newark,DE,DE0016,Taylor Deemer
12 -86.9324895189999,40.443196803,15,Purdue University,West Lafayette,IN,IN0100,Josh Hall
13 -96.586818358,39.190145864,17,Kansas State University,Manhattan,KS,KS0111,John Ritchey
14 -120.663191003,35.2743091450001,19,California Polytechnic State University,San Luis Obispo,CA,CA0355,Derek Swindall
15 -92.6404638509999,32.5296733270001,21,Louisiana Tech University,Ruston,LA,LA0147,Tommy Jackson
16 -92.453318655,35.0873368760001,23,University of Central Arkansas,Conway,AR,AR0124,Bob Butler
17 -94.1609101889999,36.076382166,24,U of A Razorback,Fayetteville,AR,AR0110,Iye Anderson
18 -91.1041724509999,30.4122830380001,25,Louisiana State University,Baton Rouge,LA,LA0005,Zac Brown
19 -131.835462245,30.7399817620001,26,Chico State University,Chico,CA,CA0380,Brian Henderson
20 -90.465618317,30.504446397,28,Southeast Louisiana University,Hammond,LA,LA0157,Zac Brown
21 -105.078300374,40.559236561,29,Colorado State University,Fort Collins,CO,CO0108,Erik Wettersten
22 -93.2085369069999,30.214655572,30,McNeese State University,Lake Charles,LA,LA0159,Derrick Davis
23 -84.5585235189999,42.7213652670001,31,Michigan State University,Lansing,MI,MI0010,Craig Curtis
```

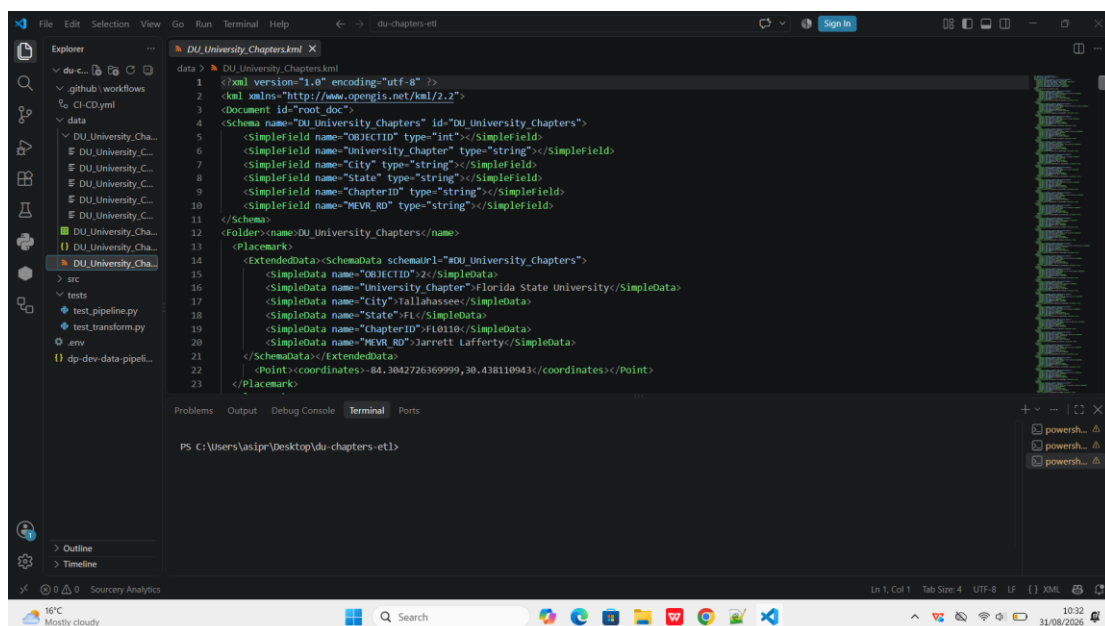
The terminal at the bottom shows the command 'PS C:\Users\asip\Desktop\du-chapters-eti>'.

JSON:-



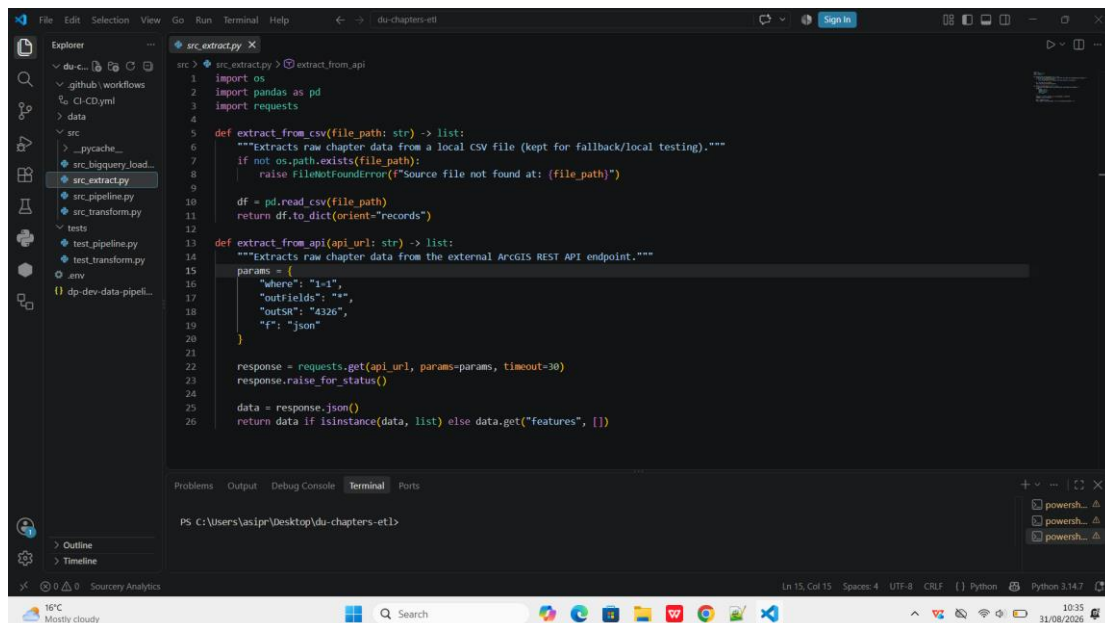
```
1 {
2   "type": "FeatureCollection",
3   "name": "DU_University_Chapters",
4   "crs": { "type": "name", "properties": { "name": "urn:ogc:def:crs:OGC:1.3:CRS84" } },
5   "features": [
6     { "type": "Feature", "properties": { "OBJECTID": 2, "University_Chapter": "Florida State University", "City": "Tallahassee", "State": "FL", "ChapterID": "FL001" }, "geometry": {} },
7     { "type": "Feature", "properties": { "OBJECTID": 3, "University_Chapter": "Abraham Baldwin Agricultural College", "City": "Tifton", "State": "GA", "ChapterID": "GA001" }, "geometry": {} },
8     { "type": "Feature", "properties": { "OBJECTID": 4, "University_Chapter": "University of Georgia", "City": "Athens", "State": "GA", "ChapterID": "GA002" }, "geometry": {} },
9     { "type": "Feature", "properties": { "OBJECTID": 5, "University_Chapter": "Georgia Southern University", "City": "Statesboro", "State": "GA", "ChapterID": "GA003" }, "geometry": {} },
10    { "type": "Feature", "properties": { "OBJECTID": 6, "University_Chapter": "Georgia Tech University", "City": "Atlanta", "State": "GA", "ChapterID": "GA004" }, "geometry": {} },
11    { "type": "Feature", "properties": { "OBJECTID": 7, "University_Chapter": "University of Alabama", "City": "Tuscaloosa", "State": "AL", "ChapterID": "AL001" }, "geometry": {} },
12    { "type": "Feature", "properties": { "OBJECTID": 9, "University_Chapter": "University of Auburn", "City": "Auburn", "State": "AL", "ChapterID": "AL002" }, "geometry": {} },
13    { "type": "Feature", "properties": { "OBJECTID": 11, "University_Chapter": "Iowa State University", "City": "Ames", "State": "IA", "ChapterID": "IA001" }, "geometry": {} },
14    { "type": "Feature", "properties": { "OBJECTID": 12, "University_Chapter": "Southern Illinois University", "City": "Carbondale", "State": "IL", "ChapterID": "IL001" }, "geometry": {} },
15    { "type": "Feature", "properties": { "OBJECTID": 13, "University_Chapter": "University of Delaware", "City": "Newark", "State": "DE", "ChapterID": "DE001" }, "geometry": {} },
16    { "type": "Feature", "properties": { "OBJECTID": 15, "University_Chapter": "Purdue University", "City": "West Lafayette", "State": "IN", "ChapterID": "IN001" }, "geometry": {} },
17    { "type": "Feature", "properties": { "OBJECTID": 17, "University_Chapter": "Kansas State University", "City": "Manhattan", "State": "KS", "ChapterID": "KS001" }, "geometry": {} },
18    { "type": "Feature", "properties": { "OBJECTID": 19, "University_Chapter": "California Polytechnic State University", "City": "San Luis Obispo", "State": "CA", "ChapterID": "CA001" }, "geometry": {} },
19    { "type": "Feature", "properties": { "OBJECTID": 21, "University_Chapter": "Louisiana Tech University", "City": "Ruston", "State": "LA", "ChapterID": "LA001" }, "geometry": {} },
20    { "type": "Feature", "properties": { "OBJECTID": 23, "University_Chapter": "University of Central Arkansas", "City": "Conway", "State": "AR", "ChapterID": "AR001" }, "geometry": {} },
21    { "type": "Feature", "properties": { "OBJECTID": 24, "University_Chapter": "U of A Razorback", "City": "Fayetteville", "State": "AR", "ChapterID": "AR002" }, "geometry": {} },
22    { "type": "Feature", "properties": { "OBJECTID": 25, "University_Chapter": "Louisiana State University", "City": "Baton Rouge", "State": "LA", "ChapterID": "LA002" }, "geometry": {} },
23    { "type": "Feature", "properties": { "OBJECTID": 26, "University_Chapter": "Chico State University", "City": "Chico", "State": "CA", "ChapterID": "CA002" }, "geometry": {} }
24  ]
25 }
```

KML:-



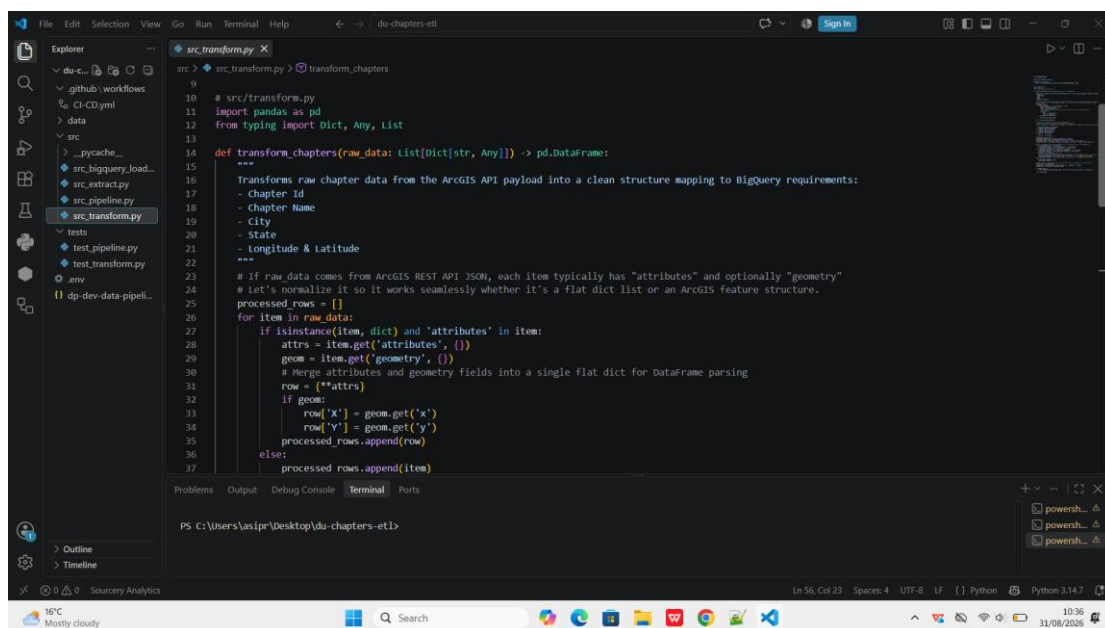
```
1 <?xml version="1.0" encoding="utf-8" ?>
2 <kml xmlns="http://www.opengis.net/kml/2.2">
3   <document id="root_doc">
4     <Schema name="DU_University_Chapters" id="DU_University_Chapters">
5       <SimpleField name="OBJECTID" type="int"/></SimpleField>
6       <SimpleField name="University_Chapter" type="string"/></SimpleField>
7       <SimpleField name="City" type="string"/></SimpleField>
8       <SimpleField name="State" type="string"/></SimpleField>
9       <SimpleField name="ChapterID" type="string"/></SimpleField>
10      <SimpleField name="MEVR_RD" type="string"/></SimpleField>
11    </Schema>
12    <Folder name="DU_University_Chapters">
13      <Placemark>
14        <ExtendedData><SchemaData schemaUri="#DU_University_Chapters">
15          <SimpleData name="OBJECTID">2</SimpleData>
16          <SimpleData name="University_Chapter">Florida State University</SimpleData>
17          <SimpleData name="City">Tallahassee</SimpleData>
18          <SimpleData name="State">FL</SimpleData>
19          <SimpleData name="ChapterID">FL001</SimpleData>
20          <SimpleData name="MEVR_RD">Jarrett Lafferty</SimpleData>
21        </SchemaData></ExtendedData>
22        <Point><coordinates>-84.30427263699999,30.438110943</coordinates></Point>
23      </Placemark>
24    </Folder>
25  </document>
26 </kml>
```

Step 3:- Below code reads raw chapter records from a local file as a fallback or testing option. Connects to the external ArcGIS REST API using `requests` with parameters to fetch raw JSON feature data.



```
1 import os
2 import pandas as pd
3 import requests
4
5 def extract_from_csv(file_path: str) -> list:
6     """Extracts raw chapter data from a local CSV file (kept for fallback/local testing)."""
7     if not os.path.exists(file_path):
8         raise FileNotFoundError(f"Source file not found at: {file_path}")
9
10    df = pd.read_csv(file_path)
11    return df.to_dict(orient="records")
12
13 def extract_from_api(api_url: str) -> list:
14     """Extracts raw chapter data from the external ArcGIS REST API endpoint."""
15     params = {
16         "where": "1=1",
17         "outfields": "**",
18         "outSR": "4326",
19         "f": "json"
20     }
21
22     response = requests.get(api_url, params=params, timeout=30)
23     response.raise_for_status()
24
25     data = response.json()
26     return data if isinstance(data, list) else data.get("features", [])
```

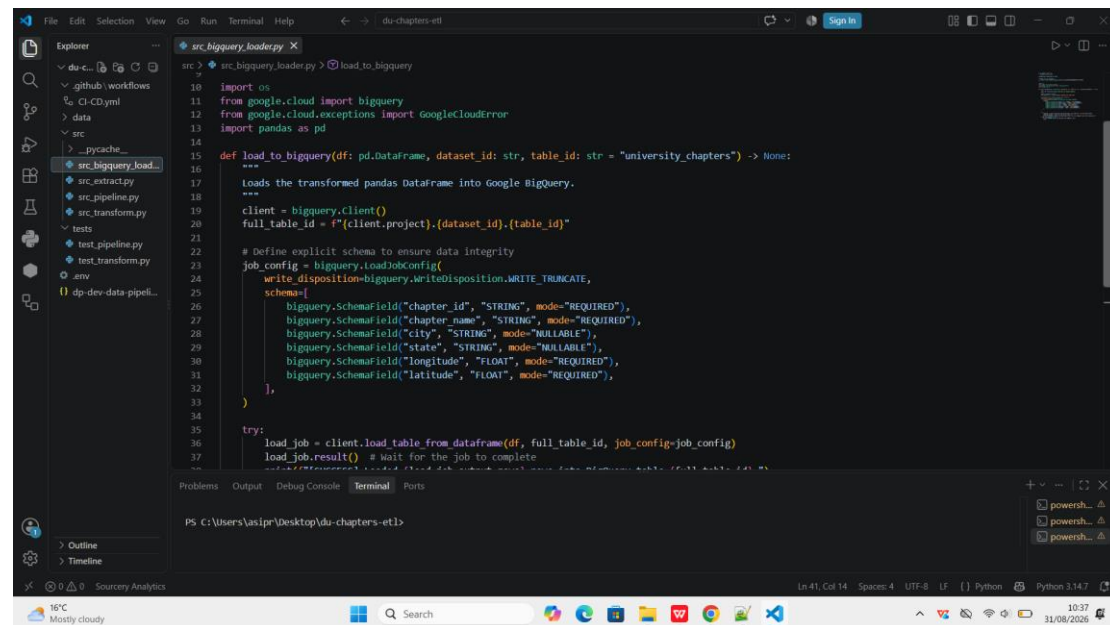
Step 4:- Normalizes raw API payloads by separating or parsing nested `attributes` and spatial `geometry` fields. Extracts longitude (`x`) and latitude (`y`) coordinates to map structural requirements for downstream storage. Outputs a clean Pandas DataFrame mapping core fields like Chapter ID, Name, City, and State.



```
1 # src/transform.py
2 import pandas as pd
3 from typing import Dict, Any, List
4
5 def transform_chapters(raw_data: List[Dict[str, Any]]) -> pd.DataFrame:
6     """
7     Transforms raw chapter data from the ArcGIS API payload into a clean structure mapping to BigQuery requirements:
8     - Chapter ID
9     - Chapter Name
10    - City
11    - State
12    - Longitude & Latitude
13    """
14    # If raw_data comes from ArcGIS REST API JSON, each item typically has "attributes" and optionally "geometry"
15    # Let's normalize it so it works seamlessly whether it's a flat dict list or an ArcGIS feature structure.
16    processed_rows = []
17    for item in raw_data:
18        if isinstance(item, dict) and 'attributes' in item:
19            attrs = item.get('attributes', {})
20            geom = item.get('geometry', {})
21            # Merge attributes and geometry fields into a single flat dict for DataFrame parsing
22            row = {"**attrs": **attrs}
23            if geom:
24                row['x'] = geom.get('x')
25                row['y'] = geom.get('y')
26            processed_rows.append(row)
27        else:
28            processed_rows.append(item)
29    return pd.DataFrame(processed_rows)
```

Step 5:- In the below code:

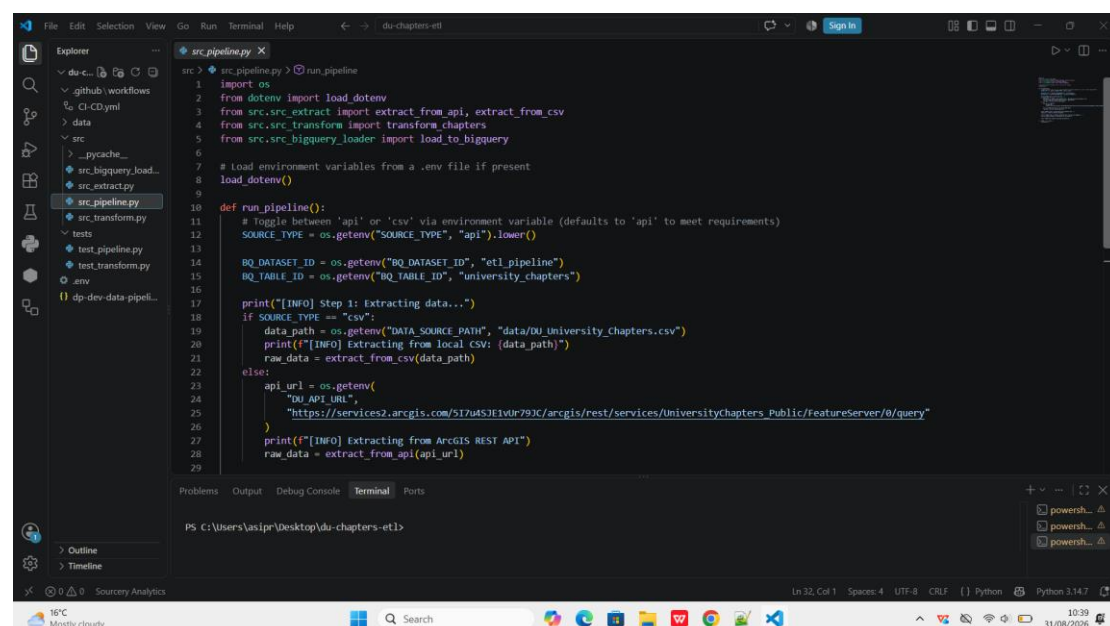
- Client Connection: Initializes a Google Cloud BigQuery client to connect to your project.
- Schema Enforcement: Defines strict data types and constraints ('STRING', 'FLOAT', 'REQUIRED', 'NULLABLE') for all chapter attributes.
- Table Overwrite: Loads the Pandas DataFrame into BigQuery using 'WRITE_TRUNCATE' to refresh the target table.



```
src > src.bigquery_loader.py > load_to_bigquery
10
11 import os
12 from google.cloud import bigquery
13 from google.cloud.exceptions import GoogleCloudError
14 import pandas as pd
15
16 def load_to_bigquery(df: pd.DataFrame, dataset_id: str, table_id: str = "university_chapters") -> None:
17     """
18     Loads the transformed pandas DataFrame into Google BigQuery.
19     """
20     client = bigquery.Client()
21     full_table_id = f"{client.project}.{dataset_id}.{table_id}"
22
23     # Define explicit schema to ensure data integrity
24     job_config = bigquery.LoadJobConfig(
25         write_disposition=bigquery.WriteDisposition.WRITE_TRUNCATE,
26         schema=[
27             bigquery.SchemaField("chapter_id", "STRING", mode="REQUIRED"),
28             bigquery.SchemaField("chapter_name", "STRING", mode="REQUIRED"),
29             bigquery.SchemaField("city", "STRING", mode="NULLABLE"),
30             bigquery.SchemaField("state", "STRING", mode="NULLABLE"),
31             bigquery.SchemaField("longitude", "FLOAT", mode="REQUIRED"),
32             bigquery.SchemaField("latitude", "FLOAT", mode="REQUIRED"),
33         ],
34     )
35
36     try:
37         load_job = client.load_table_from_dataframe(df, full_table_id, job_config=job_config)
38         load_job.result() # Wait for the job to complete
39     except GoogleCloudError as e:
40         print(f"Error loading data to BigQuery: {e}")
41
42 if __name__ == "__main__":
43     # Example usage: Load a DataFrame from a local file
44     df = pd.read_csv("data/university_chapters.csv")
45     load_to_bigquery(df, "dataset_id")
```

Step 6:- In below pipeline file:-

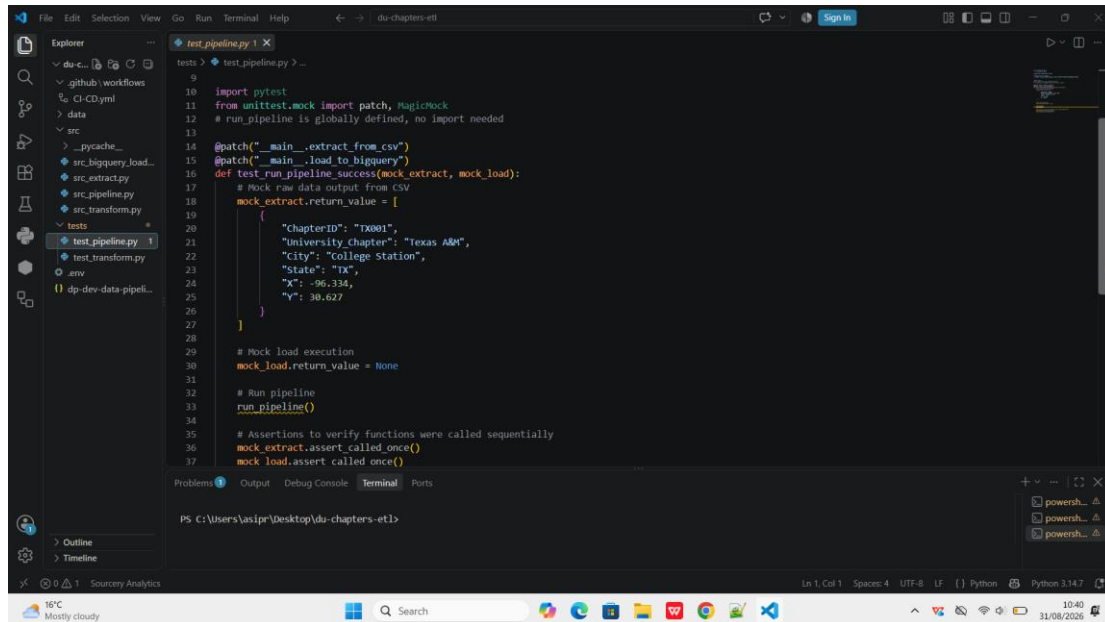
- Environment Configuration: Loads environment variables using 'load_dot_env()' and sets configurations like data source type ('api' or 'csv'), dataset ID, and table ID.
- Orchestration Execution: Acts as the master control script that sequentially coordinates the workflow by calling the extraction, transformation, and loading functions.



```
src > src.pipeline.py > run_pipeline
1
2 import os
3 from dotenv import load_dotenv
4 from src.src_extract import extract_from_api, extract_from_csv
5 from src.src_transform import transform_chapters
6 from src.src_bigquery_loader import load_to_bigquery
7
8 # Load environment variables from a .env file if present
9 load_dotenv()
10
11 def run_pipeline():
12     # Toggle between 'api' or 'csv' via environment variable (defaults to 'api' to meet requirements)
13     SOURCE_TYPE = os.getenv("SOURCE_TYPE", "api").lower()
14
15     BQ_DATASET_ID = os.getenv("BQ_DATASET_ID", "etl_pipeline")
16     BQ_TABLE_ID = os.getenv("BQ_TABLE_ID", "university_chapters")
17
18     print(f"[INFO] Step 1: Extracting data...")
19     if SOURCE_TYPE == "csv":
20         data_path = os.getenv("DATA_SOURCE_PATH", "data/du_university_chapters.csv")
21         print(f"[INFO] Extracting from local CSV: {data_path}")
22         raw_data = extract_from_csv(data_path)
23     else:
24         api_url = os.getenv(
25             "DU_API_URL",
26             "https://services2.arcgis.com/S17u4SJE1vUr793C/arcgis/rest/services/Universitychapters_Public/featureserver/0/query"
27         )
28         print(f"[INFO] Extracting from ArcGIS REST API")
29         raw_data = extract_from_api(api_url)
```


Step 7:- In testing

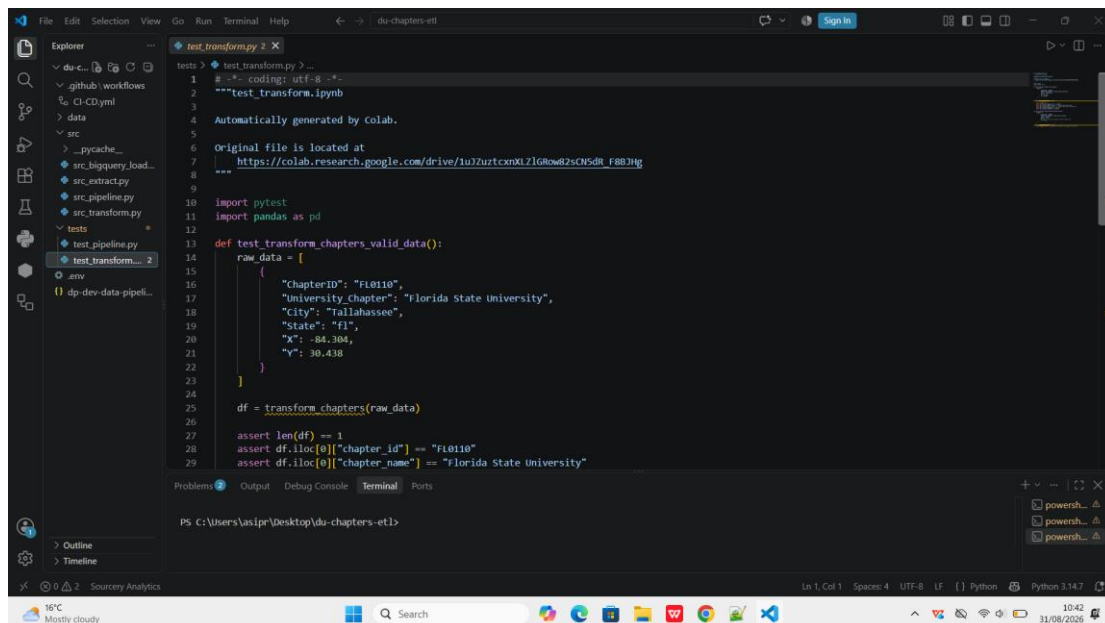
- Pipeline Mocking: Uses `unittest.mock` (`patch`) to isolate the orchestration test by mocking external dependencies like extraction and loading functions.
- Execution & Assertion: Simulates a successful run by passing mock chapter data, executing `run\_pipeline()`, and asserting that both extract and load steps were called sequentially.



```
9
10 import pytest
11 from unittest.mock import patch, MagicMock
12 # run_pipeline is globally defined, no import needed
13
14 @patch("__main__._extract_from_csv")
15 @patch("__main__._load_to_bigquery")
16 def test_run_pipeline_success(mock_extract, mock_load):
17     # Mock raw data output from CSV
18     mock_extract.return_value = [
19         {
20             "ChapterID": "TX001",
21             "University_Chapter": "Texas A&M",
22             "City": "College Station",
23             "State": "TX",
24             "X": -96.334,
25             "Y": 30.627
26         }
27     ]
28
29     # Mock load execution
30     mock_load.return_value = None
31
32     # Run pipeline
33     run_pipeline()
34
35     # Assertions to verify functions were called sequentially
36     mock_extract.assert_called_once()
37     mock_load.assert_called_once()
```

Step 8:-

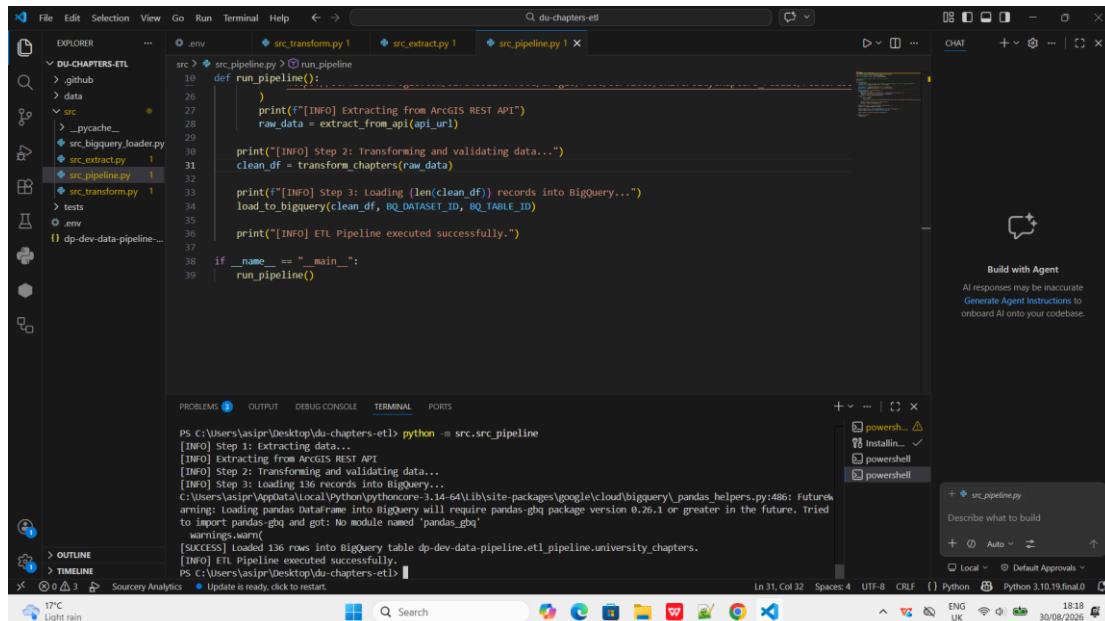
- Transformation Testing**: Tests the `transform\_chapters` function using sample raw chapter data to verify proper parsing.
- Assertion Checks**: Validates that the resulting DataFrame has the correct row count (`len(df) == 1`) and accurately maps specific fields like chapter ID and chapter name.



```
1 # -*- coding: utf-8 -*-
2 """test_transform.ipynb
3
4 Automatically generated by colab.
5
6 Original file is located at
7 https://colab.research.google.com/drive/1uJ2urtxnXlZ1GRow82sCNSdR_F8B3Hg
8 """
9
10 import pytest
11 import pandas as pd
12
13 def test_transform_chapters_valid_data():
14     raw_data = [
15         {
16             "ChapterID": "FL0110",
17             "University_Chapter": "Florida State University",
18             "City": "Tallahassee",
19             "State": "FL",
20             "X": -84.384,
21             "Y": 30.438
22         }
23     ]
24
25     df = transform_chapters(raw_data)
26
27     assert len(df) == 1
28     assert df.iloc[0]["chapter_id"] == "FL0110"
29     assert df.iloc[0]["chapter_name"] == "Florida State University"
```

Step 9:- When I run command in terminal:-

Python -m src.src_pipeline it will load 136 records in my BigQuery table(university_chapters) after processing



```
def run_pipeline():
    print(f"[INFO] Extracting from ArcGIS REST API")
    raw_data = extract_from_api(api_url)

    print(f"[INFO] Step 2: Transforming and validating data...")
    clean_df = transform_chapters(raw_data)

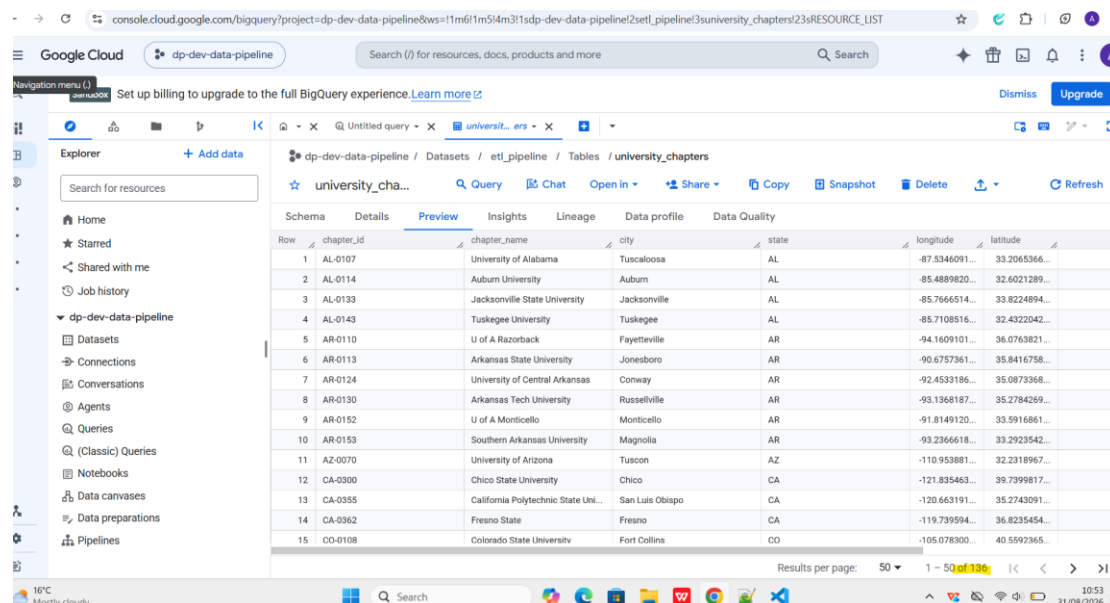
    print(f"[INFO] Step 3: Loading {len(clean_df)} records into BigQuery...")
    load_to_bigquery(clean_df, BQ_DATASET_ID, BQ_TABLE_ID)

    print(f"[INFO] ETL Pipeline executed successfully.")

if __name__ == "__main__":
    run_pipeline()
```

```
PS C:\Users\vasip\Desktop\da-chapters-etl> python -m src.src_pipeline
[INFO] Step 1: Extracting data...
[INFO] Extracting from ArcGIS REST API
[INFO] Step 2: Transforming and validating data...
[INFO] Step 3: Loading 136 records into BigQuery...
C:\Users\vasip\AppData\Local\Python\Pythoncore-3.14-64\Lib\site-packages\google\cloud\bigquery\pandas_helpers.py:486: FutureWarning: loading pandas Dataframe into BigQuery will require pandas-gbq package version 0.26.1 or greater in the future. Tried to import pandas-gbq and got: No module named 'pandas_gbq'
[SUCCESS] loaded 136 rows into BigQuery table dp-dev-data-pipeline.etl.pipeline.university_chapters.
[INFO] ETL Pipeline executed successfully.
PS C:\Users\vasip\Desktop\da-chapters-etl>
```

Step 10:- Total 136 records were loaded in BigQuery table



Row	chapter_id	chapter_name	city	state	longitude	latitude
1	AL-0107	University of Alabama	Tuscaloosa	AL	-87.5346091...	33.2065366...
2	AL-0114	Auburn University	Auburn	AL	-85.4889820...	32.6021289...
3	AL-0133	Jacksonville State University	Jacksonville	AL	-85.7666514...	33.8224894...
4	AL-0143	Tuskegee University	Tuskegee	AL	-85.7108516...	32.4322042...
5	AR-0110	U of A Razorback	Fayetteville	AR	-94.1609101...	36.0763821...
6	AR-0113	Arkansas State University	Jonesboro	AR	-90.6757361...	35.8416758...
7	AR-0124	University of Central Arkansas	Conway	AR	-92.4533186...	35.0873368...
8	AR-0130	Arkansas Tech University	Russellville	AR	-93.1368187...	35.2784269...
9	AR-0152	U of A Monticello	Monticello	AR	-91.8149120...	33.5916861...
10	AR-0153	Southern Arkansas University	Magnolia	AR	-93.2366618...	33.2923542...
11	AZ-0070	University of Arizona	Tucson	AZ	-110.953881...	32.2318967...
12	CA-0300	Chico State University	Chico	CA	-121.835463...	39.7399817...
13	CA-0355	California Polytechnic State Uni...	San Luis Obispo	CA	-120.663191...	35.2743091...
14	CA-0362	Fresno State	Fresno	CA	-119.739594...	36.8235454...
15	CO-0108	Colorado State University	Fort Collins	CO	-105.078300...	40.5892365...